

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
SIMATS ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
CSA17 - ARTIFICIAL INTELLIGENCE

Experiment No. 7: Python Program to Implement Breadth-First Search (BFS)

Aim

To write a Python program to implement the Breadth-First Search (BFS) algorithm for traversing a graph, exploring all the neighbors of a node before moving to the next level of nodes, using a queue-based data structure.

Algorithm

1. Start.
2. Represent the graph as an adjacency list using a dictionary, where each key is a node and its value is the list of adjacent (neighboring) nodes.
3. Read the starting node from which the traversal should begin.
4. Create an empty queue and insert the starting node into it. Create a visited set and mark the starting node as visited.
5. Create an empty list 'order' to store the sequence in which nodes are visited.
6. While the queue is not empty: a) Remove (dequeue) the front node from the queue and append it to 'order'. b) For each neighbor of this node in the adjacency list: if the neighbor has not been visited, mark it visited and insert it into the queue.
7. Repeat step 6 until the queue becomes empty, meaning all reachable nodes have been visited.
8. Print the final BFS traversal order.
9. Stop.

Pseudocode

```
BEGIN
  INPUT: graph (adjacency list), start_node
  CREATE queue Q, ENQUEUE start_node
  CREATE VISITED = { start_node }
  CREATE order = []

  WHILE Q is not empty DO
    node = Q.dequeue()
    APPEND node to order

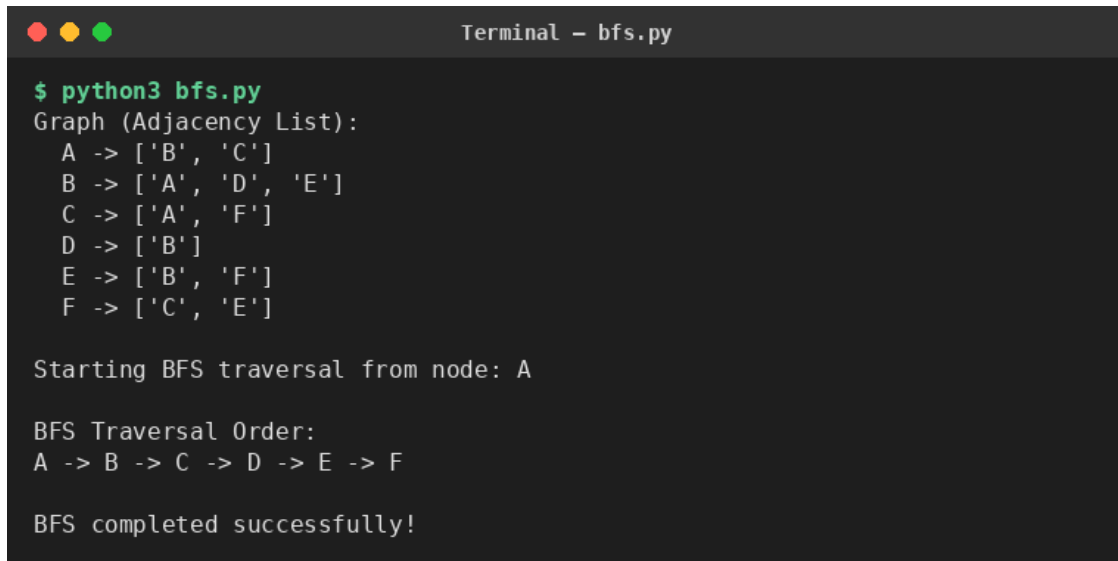
    FOR each neighbor IN graph[node] DO
      IF neighbor NOT IN VISITED THEN
        ADD neighbor TO VISITED
        Q.enqueue(neighbor)
      END IF
    END FOR
  END WHILE

  PRINT order
END
```

Result

The Python program to implement Breadth-First Search was executed successfully. For the given graph with nodes A through F, starting the traversal from node A, the program visited all reachable nodes level by level using a queue, producing the traversal order A -> B -> C -> D -> E -> F. Thus, the Breadth-First Search algorithm was implemented and verified successfully using Python.

Output Screenshot:

A terminal window titled "Terminal - bfs.py" with a dark background and light green text. It shows the execution of a Python script. The output displays the graph structure as an adjacency list, the starting node for BFS, the resulting traversal order, and a confirmation that the search was completed successfully.

```
Terminal - bfs.py

$ python3 bfs.py
Graph (Adjacency List):
A -> ['B', 'C']
B -> ['A', 'D', 'E']
C -> ['A', 'F']
D -> ['B']
E -> ['B', 'F']
F -> ['C', 'E']

Starting BFS traversal from node: A

BFS Traversal Order:
A -> B -> C -> D -> E -> F

BFS completed successfully!
```